ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

The expressive power of the Lambda Calculus

We closed our last lecture mentioning the Church-Turing Thesis about the different characterizations of the notion of computable functions. A you (should) know, the thesis has remained without a proof, but the various cross-encodings that proved the equivalent expressive power of lambda-calculus and other models of computation has long been considered as a *de-facto* proof.

Instead of discussing such cross-encodings, below we illustrate the power of the Lambda Calculus by first howing it at work on the encoding of various concepts and mechanisms found in modern programming langugage. In particular, we look at the encoding of

- simple datatypes - numerals and booleans - together with their operators
- recursion

Based on that, we then show that the λ calculus is powerful enough to encode the Kleene's *Partial Recursive Functions*

Church Booleans

We start with Booleans and conditional expressions.

$$\begin{array}{rcl} \mathsf{true} & = & \lambda x.\lambda y.x \\ \mathsf{false} & = & \lambda x.\lambda y.y \\ \mathsf{cond} & = & \lambda x.\lambda y.\lambda z(x\,y\,z) \end{array}$$

As we see, true and false are represented as functions that take two arguments and return, respectively, the first and the second argument.

Though the definitions may appear surprising at first, you can make sense of them by looking at the encoding of the conditional and verifying that, whenever M is either true and false:

$$\mathsf{cond}\,M\,N\,P \quad \text{behaves as} \quad \mathsf{if}\ M\ \mathsf{then}\ N\ \mathsf{else}\ P$$

Given that true and false choose their first and second argument, respectively, they may be combined to represent the logical operator similarly to how they are employed to select the branches of the conditional.

$$\begin{aligned}
\mathsf{and} &= \lambda x.\lambda y.x\ y\ \mathsf{false} \\
\mathsf{or} &= \lambda x.\lambda y.x\ \mathsf{true}\ y \\
\mathsf{not} &= \lambda p.p\ \mathsf{false}\ \mathsf{true}
\end{aligned}$$

We look at the definitions at work on few examples:

$$\begin{aligned}
\mathsf{and}\ \mathsf{true}\ \mathsf{false} &\to\to \mathsf{true}\ \mathsf{false}\ \mathsf{false} \\
&\to\to \mathsf{false} \\
\mathsf{or}\ \mathsf{true}\ \mathsf{false} &\to\to \mathsf{true}\ \mathsf{true}\ \mathsf{false} \\
&\to\to \mathsf{true} \\
\mathsf{not}\ \mathsf{true} &\to\to \mathsf{true}\ \mathsf{false}\ \mathsf{true} \\
&\to\to \mathsf{false}
\end{aligned}$$

Church Numerals

We show the encoding of integers by means of *Church numerals*. The underlying intuition is to use the structure of lambda terms to *count*.

Each number is represented as a lambda abstraction that expects two arguments — a function and an argument — and applies the function a corresponding number of times. Thus, given any function f and argument x, we have:

$$\begin{aligned}
&\mathsf{0}\ \text{applies } f \text{ to } x \text{ zero times}: && \mathsf{0}\ f\ x = x \\
&\mathsf{1}\ \text{applies } f \text{ to } x \text{ one time}: && \mathsf{1}\ f\ x = f\ x \\
&\mathsf{2}\ \text{applies } f \text{ to } x \text{ two times}: && \mathsf{2}\ f\ x = f(f\ x) \\
&\mathsf{3}\ \text{applies } f \text{ to } x \text{ two times}: && \mathsf{3}\ f\ x = f(f(f\ x)) \\
&\qquad \dots && \qquad \dots \\
&\mathsf{n}\ \text{applies } f \text{ to } x\ n \text{ times}: && \mathsf{n}\ f\ x = f^n\ x
\end{aligned}$$

The encoding can therefore be defined as follows:

$$
\begin{array}{lcl}
0 & = & \lambda \mathsf{f}.\lambda \mathsf{x}.\mathsf{x} \\
1 & = & \lambda \mathsf{f}.\lambda \mathsf{x}.(\mathsf{f}\,\mathsf{x}) \\
2 & = & \lambda \mathsf{f}.\lambda \mathsf{x}.\mathsf{f}(\mathsf{f}\,\mathsf{x}) \\
3 & = & \lambda \mathsf{f}.\lambda \mathsf{x}.\mathsf{f}(\mathsf{f}(\mathsf{f}\,\mathsf{x})) \\
\vdots & & \vdots \\
\mathsf{n} & = & \lambda \mathsf{f}.\lambda \mathsf{x}.(\mathsf{f}^{\mathsf{n}}\,\mathsf{x})
\end{array}
$$

The arithmetic operators can be represented as functions over Church numerals.

- **Successor.** We need $n+1$ applications of f Since $\mathsf{n}\,\mathsf{f}\,\mathsf{x}$ gives us n applications, one more does the trick:

  $$\mathsf{succ} = \lambda \mathsf{n}.\lambda \mathsf{f}.\lambda \mathsf{x}.\mathsf{f}\,(\mathsf{n}\,\mathsf{f}\,\mathsf{x})$$

- **Plus**. We need $m+n$ applications of f Start with n ($\mathsf{n}\,\mathsf{f}\,\mathsf{x}$), then apply f another m times on top:

  $$\mathsf{plus} = \lambda \mathsf{m}.\lambda \mathsf{n}.\lambda \mathsf{f}.\lambda \mathsf{x}.\mathsf{m}\,\mathsf{f}(\mathsf{n}\,\mathsf{f}\,\mathsf{x})$$

- **Mult**. We need $m \times n$ applications of f. The term $\mathsf{n}\,\mathsf{f}$ is a function that applies f n times; applying this function m times gives $m \times n$applications:

  $$\mathsf{mult} = \lambda \mathsf{m}.\lambda \mathsf{n}.\lambda \mathsf{f}.\lambda \mathsf{x}.\mathsf{m}\,(\mathsf{n}\,\mathsf{f})\,\mathsf{x}$$

Recursion

Here's a puzzle: how do you write recursive functions in the λ calculus? There's no `let rec` , no function names to refer back to—functions are anonymous!

The answer lies in the untyped nature of the λ calculus and the power of **self-application**. Since every term denotes a function, and functions can be applied to anything, functions can be applied to *themselves*.

The Y Combinator Haskell Curry discovered a remarkable term called the **Y combinator**:

$$\mathsf{Y} = \lambda \mathsf{f}.(\lambda \mathsf{x}.\mathsf{f}\,(\mathsf{x}\,\mathsf{x}))(\lambda \mathsf{x}.\mathsf{f}\,(\mathsf{x}\,\mathsf{x}))$$

This combinator computes **fixed points**.

Fixpoint Theorem. Every λ term F has a fixpoint—a term M such that $\mathsf{M} \to^* \mathsf{F}\,\mathsf{M}$.

Proof. Take $\mathsf{M} = \mathsf{Y}\,\mathsf{F}$. Then:

$$\begin{aligned}
\mathsf{Y}\,\mathsf{F} &= (\lambda \mathsf{f}.(\lambda \mathsf{x}.\mathsf{f}\,(\mathsf{x}\,\mathsf{x}))(\lambda \mathsf{x}.\mathsf{f}\,(\mathsf{x}\,\mathsf{x})))\,\mathsf{F} \\
&\to (\lambda \mathsf{x}.\mathsf{F}\,(\mathsf{x}\,\mathsf{x}))(\lambda \mathsf{x}.\mathsf{F}\,(\mathsf{x}\,\mathsf{x})) \\
&\to \mathsf{F}\,((\lambda \mathsf{x}.\mathsf{F}\,(\mathsf{x}\,\mathsf{x}))(\lambda \mathsf{x}.\mathsf{F}\,(\mathsf{x}\,\mathsf{x}))) \\
&= \mathsf{F}\,(\mathsf{Y}\,\mathsf{F}) \quad \square
\end{aligned}$$

How does this help with recursion? Consider the factorial function: To see how, consider a the standard recursive definition of the factorial function:

```haskell
-- haskell
fact = \n -> if (n == 0) 1 else n * fact(n - 1)
```

We are so accustomed to such definitions that we tend to overlook what they really say. This is not a function definition in the usual sense — it is an *equation*, with fact appearing on both sides.
In other words, the definition implicitly characterizes fact as the solution to this equation: a function f such that $f = F\,f$, where

$$\mathsf{F} = \lambda \mathsf{f}.\lambda \mathsf{n}.\,\text{if}\,(\mathsf{n} = 0)\,1\,\text{else}\,\mathsf{n} \times \mathsf{f}(\mathsf{n} - 1)$$

The Y combinator computes exactly this: $Y\,F$ is a fixpoint of F, thus we may defined

$$\mathsf{fact} = \mathsf{Y}\,\mathsf{F}$$

The Y combinator "ties the knot"—it feeds the function to itself, enabling recursion without explicit self-reference.

We can verify that this captures the expected behavior

$$\begin{aligned}
\mathsf{fact} &= \mathsf{Y}\,\mathsf{F} && (\text{by definition}) \\
&\to^* \mathsf{F}(\mathsf{Y}\,\mathsf{F}) && (\text{by the fixpoint theorem}) \\
&= \mathsf{F}\,\mathsf{fact} && (\text{by definition}) \\
&\to \text{if}\ (\mathsf{n} = 0)\,1\,\text{else}\ \mathsf{n} \times \mathsf{fact}(\mathsf{n} - 1)
\end{aligned}$$

Recursion and Strict Evaluation

The Y combinator enables recursion in lambda calculus: it works perfectly with normal order.

```
λ[normal]> FACT = \f.\n.IF (ISZERO n) ONE (MULT n (f (PRED n)))
λ[normal]> :toInt (Y FACT) THREE
  6
```

But with **call-by-value** or **applicative order**, Y diverges immediately:

```
λ[value]> :toInt (Y FACT) THREE
  Warning: exceeded 1000 steps, term may diverge
```

Why? In CBV, arguments are evaluated before substitution. When we apply `Y FACT`

1. `Y FACT → (λx.FACT (x x)) (λx.FACT (x x))`
2. CBV must evaluate the argument `(λx.FACT (x x))` — it's already a value (CBV does not reduce under λ) ✓
3. Substitute → `FACT ((λx.FACT (x x)) (λx.FACT (x x)))`
4. CBV must now evaluate the argument `(λx.FACT (x x)) (λx.FACT (x x))`
5. This is `Y FACT` again! → infinite loop before FACT is ever called

Solution 1: Use the Z Combinator

The Z combinator adds an eta-expansion to delay the self-application:

```
Z = λf.(λx.f (λy.x x y)) (λx.f (λy.x x y))
```

The `λy.x x y` wrapper means `x x` isn't evaluated until it's actually needed (when `y` is supplied).

But there's still a problem: `IF` evaluates **both branches** before selecting:

```
λ[value]> FACT = \f.\n.IF (ISZERO n) ONE (MULT n (f (PRED n)))
λ[value]> (Z FACT) TWO
  Warning: exceeded 1000 steps, term may diverge
```

Even when `n = 0`, CBV evaluates `f (PRED n)` which triggers infinite recursion.

Solution 2: Use IFC with Thunks

`IFC` is a short-circuit IF that takes **thunks** (delayed values) as branches:

```
IFC = λp.λt.λf.p t f I
```

Wrap each branch in `λd.…` to delay evaluation:

```
λ[value]> FACT = \f.\n.IFC (ISZERO n) (\d.ONE) (\d.MULT n (f (PRED n)))
λ[value]> :toInt (Z FACT) THREE
  6
```

How it works:

1. `IFC` receives the condition and two thunks
2. The condition (`TRUE` or `FALSE`) selects one thunk
3. `I` is applied to force evaluation of only the selected thunk
4. The other branch is never evaluated → no infinite recursion

Summary

Strategy	Y combinator	Z combinator	Z + IFC (thunks)
normal	✓ works	✓ works	✓ works
applicative	✗ diverges	✗ diverges	✗ diverges
value	✗ diverges	needs thunks	✓ works
full	✓ works	✓ works	✓ works

Partial Recursive Functions

The theory of **partial recursive functions** (also called **μ-recursive functions**) was developed by Gödel, Herbrand, and Kleene in the 1930s as a formalization of computability based on recursion.

Building Blocks

Partial recursive functions on natural numbers are built from two **basic functions**:

- **Zero**: $Z(x) = 0$
- **Successor**: $S(x) = x + 1$

These are combined using three **operations**:

1. **Composition**: Given g and h, define $f(x) = g(h(x))$
2. **Primitive recursion**: Given g and h, define f by:

$$\begin{array}{lcl} f(0) & = & g \\ f(n+1) & = & h(n, f(n)) \end{array}$$

3. **Minimization (μ-operator)**:
   Given g, define

$f(x) = \mu y[g(x,y) = 0]$ — the smallest y such that $g(x,y) = 0$, undefined if no such y exists.

Encoding Partial Recursive Functions in the λ Calculus

We show that all the building blocks of the theory of Partial Recursive Functions can be encoded in the λ Calculus.

- The two **basic functions** have direct λ-term counterparts discussed earlier
- **Composition** is a built-in mechanism in the λ calculus
- **Primitive recursion**

$$\mathsf{F} = \mathsf{Y}\ (\lambda \mathsf{f}.\lambda \mathsf{n}.\mathsf{if}\ (\mathsf{isZero}\ \mathsf{n})\ \mathsf{g}\ (\mathsf{h}(\mathsf{pred}\ \mathsf{n})(\mathsf{f}(\mathsf{pred}\ \mathsf{n}))))$$

- **Minimization**

$$\mathsf{F} = \mathsf{Y}\ \lambda \mathsf{g}.\mathsf{Y}\ (\lambda \mathsf{s}.\lambda \mathsf{n}.\mathsf{if}\ (\mathsf{isZero}\ (\mathsf{g}\ \mathsf{n}))\ \mathsf{n}\ (\mathsf{s}(\mathsf{succ}\ \mathsf{n})))\ 0$$

Exercises

Here a few exercises on computing with λ calculus. Feel free to use the Lambda Calculus Interpreter provided on the moodle site for the course to experiment with your definitions and check your results.

1. Practice. Try and play with the definitions of the Church booleans and numerals with their operators to verify that their evaluation behaves as expected. For instance, check the following:

- $\mathsf{and}(\mathsf{not}\ \mathsf{true})\ (\mathsf{or}\ \mathsf{false}\ \mathsf{true}) = \mathsf{false}$
- $\mathsf{plus}\ 3\ 4 = 7$
- $\mathsf{mult}\ 3\ 2 = 6$
- $\ldots$

2. xor. Define the lambda calculus encoding of the logical operator XOR, which is true only when its two arguments have different truth values.

3. isZero. Define a lambda term to encode the isZero test function: given a numeral n the function returns true if n is 0, otherwise it returns false

4. Church pairs. In class we looked at a direct encoding of booleans and conditionals. Here we study a different approach that draws on pairs as the main building block.

Pairs are encoded as the following structured lambda term:

pair = λ fst. λ snd. λ f. (f fst snd)

A pair is a function that takes its two components arguments and a function which it applies to the components, so that

pair a b →∗ λ f. (f a b)

Now we can introduce the two general selectors:

select-first = λ fst. λ snd. fst
select-second = λ fst. λ snd. snd

From them, we obtain two pair selectors by applying the pair constructor to the general selectors:

first = λ p. (p *select-first*)
second = λ p. p (*select-second*)

Exercise 4.1. Verify that:
first (*pair* a b) →∗ a
second (*pair* a b) →∗ b

Exercise 4.2. Write a version of *pair* called *fun-pair* that takes five arguments including f,g,x,y and makes a pair with first element f(x) and second g(y).

5. Lists.
Lists can be encoded in the λ calculus using the same trick as Church numerals — representing data by what you can do with it.

$$\begin{aligned} \text{nil} &= \lambda c.\lambda n.\, n \\ \text{cons} &= \lambda h.\lambda t.\lambda c.\lambda n.\, c\, h\, (t\, c\, n) \end{aligned}$$

The idea: a list takes two arguments — what to do with cons cells (`c`) and what to return for nil (`n`)

Exercise 5.1 Make sure you understand these examples: specifically, for each list on the left-hand side,

- express it in terms of the two constructors nil and cons,
- shows that the resulting term reduces to the term on the right-hand side

$$
\begin{array}{lcl}
[\,] & = & \lambda c.\lambda n.\, n \\
[a] & = & \lambda c.\lambda n.\, c\, a\, n \\
[a, b] & = & \lambda c.\lambda n.\, c\, a\, (c\, b\, n) \\
[a, b, c] & = & \lambda c.\lambda n.\, c\, a\, (c\, b\, (c\, c\, n))
\end{array}
$$

Exercise 5.2: List Operations. Define the standard operations on lists based on the representation just introduced

$$
\begin{array}{lcl}
\text{head} & = & \ldots \\
\text{tail} & = & \ldots \\
\text{isNil} & = & \ldots \\
\text{length} & = & \ldots \\
\text{sum} & = & \ldots \\
\text{append} & = & \ldots
\end{array}
$$

Exercise 5.3: λ Calculus vs Programming Languages.
Now you have all the tools to play with the lambda calculus, and practice with the programming exercises proposed in the previous lectures for Haskell, Scala and Elixir.